

JavaScript Prototype Inheritance

What is Prototype Inheritance?

Prototype inheritance is a fundamental concept in JavaScript where objects can inherit properties and methods from other objects. Unlike class-based inheritance in other languages, JavaScript uses prototype-based inheritance.

Key Concepts:

- **Every object in JavaScript has a prototype**
- **Objects inherit properties and methods from their prototype**
- **If a property/method is not found in an object, JavaScript looks up the prototype chain**
- **Only objects can be inherited by objects**

The Prototype Chain

When you access a property on an object, JavaScript follows this search pattern:

1. Check if the property exists on the object itself
2. If not found, check the object's prototype (`__proto__`)
3. Continue up the prototype chain until the property is found or `null` is reached

Methods for Prototype Inheritance

1. `__proto__` (ES5 - Legacy Method)

The `__proto__` property is used to set or get the prototype of an object.

Note: This is deprecated and should be avoided in modern code.

```
const animal = {
  name: "Generic Animal",
  eat: function() {
    console.log(`${this.name} can eat`);
  }
};

const dog = {
  name: "Buddy",
  bark: function() {
    console.log(`${this.name} can bark`);
  }
};

// Setting prototype using __proto__ (deprecated)
dog.__proto__ = animal;

console.log(dog.name); // "Buddy"
dog.eat(); // "Buddy can eat" - inherited method
dog.bark(); // "Buddy can bark" - own method
```

2. `Object.setPrototypeOf()` (Modern ES6+ Method)

Syntax: `Object.setPrototypeOf(child, parent)`

- **Parameters:** Takes 2 arguments - the child object and the parent object
- **Purpose:** Sets the prototype (parent) of the specified object

```
const animal = {
  name: "Generic Animal",
  eat: function() {
    console.log(`${this.name} can eat`);
  }
};

const dog = {
  name: "Buddy",
  bark: function() {
    console.log(`${this.name} can bark`);
  }
};

// Modern way to set prototype
Object.setPrototypeOf(dog, animal);

dog.eat(); // "Buddy can eat" - inherited from animal
```

3. `Object.getPrototypeOf()`

Syntax: `Object.getPrototypeOf(object)`

- **Parameters:** Takes 1 argument - the object whose prototype you want to get
- **Purpose:** Returns the prototype (parent) of the specified object

```
const parent = Object.getPrototypeOf(dog);
console.log(parent === animal); // true
```

4. `Object.create()` (Recommended Method)

Syntax: `Object.create(prototype, properties)`

- **Parameters:**
 - `prototype`: The object to be used as prototype
 - `properties` (optional): Property descriptors for the new object

- **Purpose:** Creates a new empty object with the specified prototype

```
const earlyMan = {
  name: "Tarzan",
  hunt: function() {
    console.log(`${this.name} can hunt`);
  }
};

// Create object with prototype
const human = Object.create(earlyMan, {
  name: {
    value: "Avinash",
    writable: true,
    enumerable: true,
    configurable: true
  },
  age: {
    value: 24,
    writable: true,
    enumerable: true,
    configurable: true
  }
});

console.log(human.name); // "Avinash"
human.hunt(); // "Avinash can hunt" - inherited method
```

Property Descriptors in Object.create()

When using `Object.create()`, you can specify property descriptors:

- **value**: The value of the property
- **writable**: Whether the property can be changed (default: false)
- **enumerable**: Whether the property shows up in for...in loops (default: false)
- **configurable**: Whether the property can be deleted or modified (default: false)

Complete Inheritance Chain Example

```
// Base prototype
const animal = {
  name: "Generic Animal",
  eat: function() {
    console.log(`${this.name} can eat`);
  }
};

// Middle level
const dog = Object.create(animal, {
  name: {
    value: "Dog",
    writable: true,
    enumerable: true,
    configurable: true
  }
});

dog.bark = function() {
  console.log(`${this.name} can bark`);
};

// Top level
const superDog = Object.create(dog, {
  name: {
    value: "SuperDog",
    writable: true,
    enumerable: true,
    configurable: true
  }
});

superDog.fly = function() {
  console.log(`${this.name} can fly`);
};

// Testing the inheritance chain
superDog.fly(); // "SuperDog can fly" - own method
superDog.bark(); // "SuperDog can bark" - inherited from dog
superDog.eat(); // "SuperDog can eat" - inherited from animal
```

Other Useful Object Methods

`Object.assign()`

Copies properties from one or more source objects to a target object.

```
const user = {
  username: "avinash",
  age: 24,
  city: "Delhi"
};

const company = {
  companyName: "XYZ Info",
  jobLocation: "Gurugram"
};

const employee = Object.assign({}, user, company);
console.log(employee);
// Contains properties from both user and company objects
```

`Object.freeze()` and `Object.isFrozen()`

- `Object.freeze()` : Makes an object immutable (cannot add, delete, or modify properties)
- `Object.isFrozen()` : Checks if an object is frozen

```
const obj = {
  name: "John",
  age: 30
};

Object.freeze(obj);

obj.age = 35; // This won't work
obj.city = "NYC"; // This won't work
delete obj.name; // This won't work

console.log(Object.isFrozen(obj)); // true
console.log(obj); // Original object unchanged
```

Object.hasOwn()

Checks if an object has a specific property as its own (not inherited).

```
const obj = {  
  username: "Avinash",  
  age: 24  
};  
  
console.log(Object.hasOwn(obj, "username")); // true  
console.log(Object.hasOwn(obj, "toString")); // false (inherited from Object)
```

Best Practices

1. **Use** `Object.create()` instead of `__proto__` for setting prototypes
2. **Use** `Object.setPrototypeOf()` if you need to change prototype after object creation
3. **Use** `Object.getPrototypeOf()` instead of `__proto__` for getting prototypes
4. **Avoid modifying prototypes of built-in objects** (like Array, Object)
5. **Consider ES6 classes** for more complex inheritance patterns

Modern Alternative: ES6 Classes

While prototype inheritance is fundamental to JavaScript, ES6 classes provide a more familiar syntax:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  eat() {
    console.log(`${this.name} can eat`);
  }
}

class Dog extends Animal {
  bark() {
    console.log(`${this.name} can bark`);
  }
}

const myDog = new Dog("Buddy");
myDog.eat(); // Inherited method
myDog.bark(); // Own method
```

Summary

- Prototype inheritance allows objects to inherit from other objects
- Use modern methods like `Object.create()` and `Object.setPrototypeOf()`
- Avoid deprecated `__proto__` property
- Understanding prototypes is crucial for mastering JavaScript
- ES6 classes are built on top of prototype inheritance